

Multi-Agent Orchestration

Main Goal

Understand how an AI system can move from:

```
One LLM
  ↓
One Agent
  ↓
Agent + Multiple Tools
  ↓
Multiple Specialized Agents
  ↓
Orchestrator
  ↓
Multi-Agent System
```

The main idea is:

Instead of asking one agent to do everything, divide a complex problem into smaller responsibilities and allow specialized agents to work together.

From Agents to Multi-Agent Systems

A single agent can perform many tasks.

For example:

```
User
  ↓
AI Agent
  ↓
Search Tool
Database Tool
Calculator Tool
API Tool
```

This works well when the problem is relatively simple.

But consider a complex request:

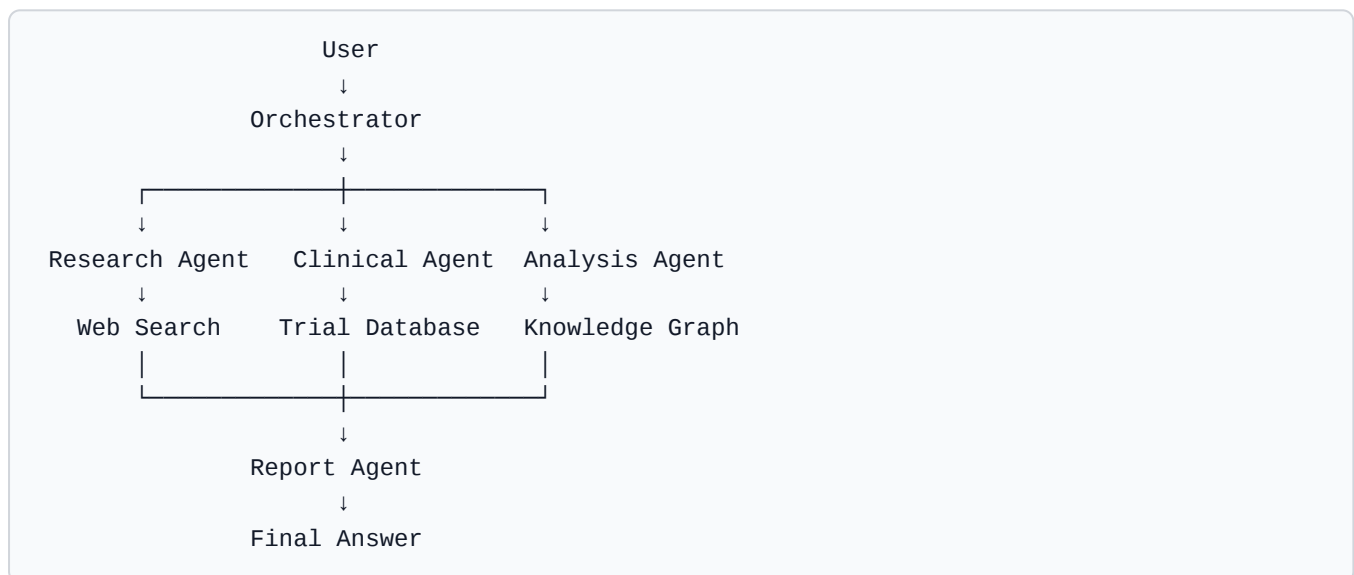
"Research a new drug, find related clinical trials, identify competing drugs, analyze research papers, and prepare a detailed report."

A single agent would need to:

- Search research papers
- Search clinical trials
- Extract entities
- Analyze relationships
- Compare drugs
- Search databases
- Generate a report
- Validate the information

This creates a very large and complex agent.

A better architecture is:



Each agent has a specific responsibility.

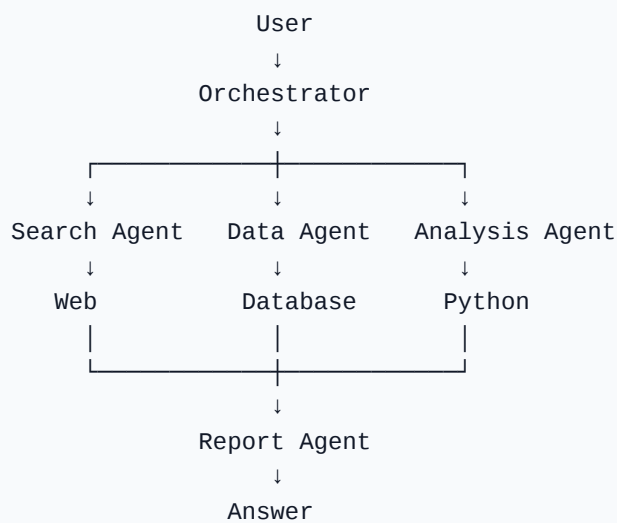
What Is a Multi-Agent System?

A Multi-Agent System is a system where multiple specialized agents collaborate to complete a larger task.

Each agent can have:

- Its own instructions
- Its own tools
- Its own model
- Its own memory
- Its own responsibilities
- Its own output format

Example:



The important concept is **specialization**.

Why Do We Need Multiple Agents?

Specialization

A single agent may be responsible for everything.

A multi-agent system divides responsibilities.

For example:

```
Research Agent
  ↓
Find information

Data Agent
  ↓
Process structured data

Analysis Agent
  ↓
Analyze information

Writer Agent
  ↓
Generate final report
```

Each agent becomes easier to design and maintain.

E-Commerce Example

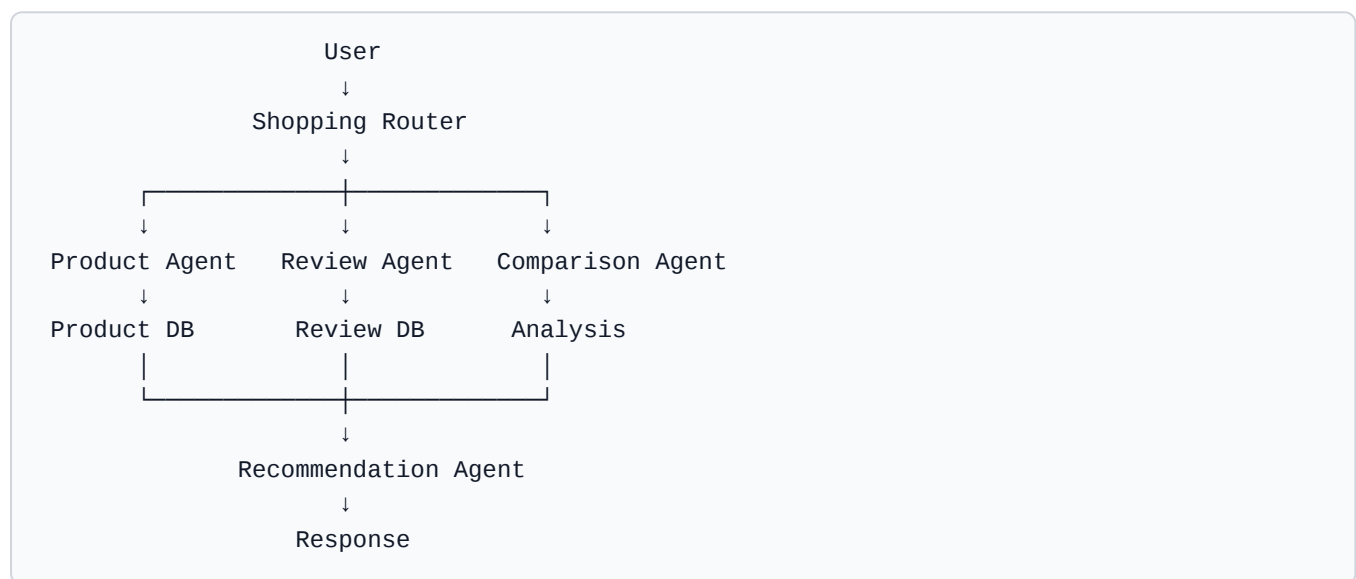
Consider an e-commerce assistant.

A user asks:

"Find me a laptop under ₹80,000, compare the specifications, check reviews, and recommend suitable options."

A single agent could perform everything.

But a multi-agent architecture could be:

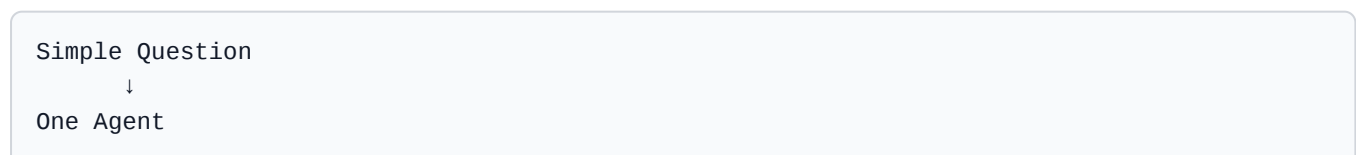


Each agent focuses on a specific responsibility.

Complexity

As the problem becomes more complex, the number of responsibilities increases.

For example:



But:

```
Research
Data Retrieval
Data Analysis
Validation
Decision Making
Report Generation
    ↓
Multiple Specialized Agents
```

Multi-agent architecture becomes useful when responsibilities are clearly separable.

Different Tools

Different agents may require completely different tools.

For example:

```
Research Agent
    → Web Search

Database Agent
    → SQL

Data Analysis Agent
    → Python

Document Agent
    → PDF Parser

Communication Agent
    → Slack / Email
```

Instead of giving every tool to every agent, tools can be isolated by responsibility.

Different Models

Not every task requires the same model.

For example:

Simple Classification
↓
Smaller / Faster Model

Complex Reasoning
↓
More Capable Model

Summarization
↓
Fast Model

Code Generation
↓
Code-optimized Model

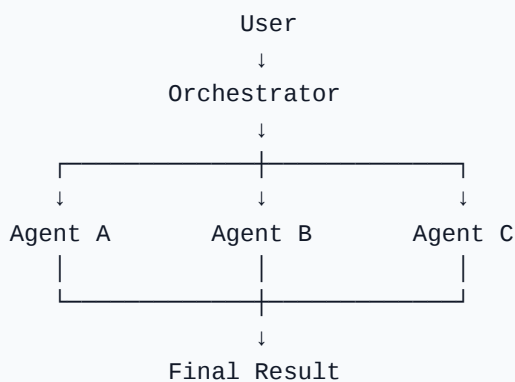
A multi-agent architecture can therefore support model specialization.

What Is Orchestration?

Orchestration means coordinating multiple agents and controlling:

- Which agent should run
- When an agent should run
- What information it receives
- What output it should produce
- Which agent should run next
- When the workflow should stop

The orchestrator acts like a coordinator.



Agent vs Orchestrator

Agent

An agent is responsible for performing a task.

Example:

Research Agent

Input:

"Find information about Alzheimer's drug trials."

Output:

Research findings

Orchestrator

The orchestrator decides which agent should perform which task.

User Request

↓

Orchestrator

↓

Research Agent

↓

Clinical Trial Agent

↓

Analysis Agent

↓

Report Agent

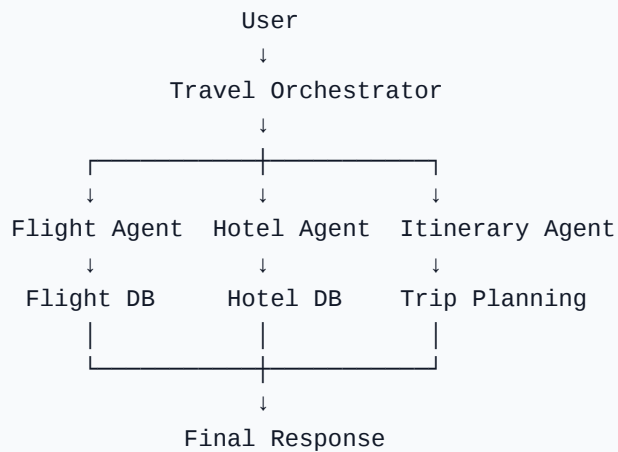
The orchestrator focuses on workflow coordination.

Travel Agent Example

Consider:

"Book me a flight from Hyderabad to Delhi, find a hotel near the airport, and prepare my itinerary."

A possible multi-agent system:



The Flight Agent handles flights.

The Hotel Agent handles hotels.

The Itinerary Agent combines the information.

Agent Specialization

Specialization means assigning a clear responsibility to each agent.

For example:

Research Agent

Responsibility:

Find relevant information.

Extraction Agent

Responsibility:

Extract entities and relationships.

Analysis Agent

Responsibility:

Analyze the extracted information.

Validation Agent

Responsibility:

Check the result.

Report Agent

Responsibility:

Generate the final response.

A good specialized agent should have a clear boundary.

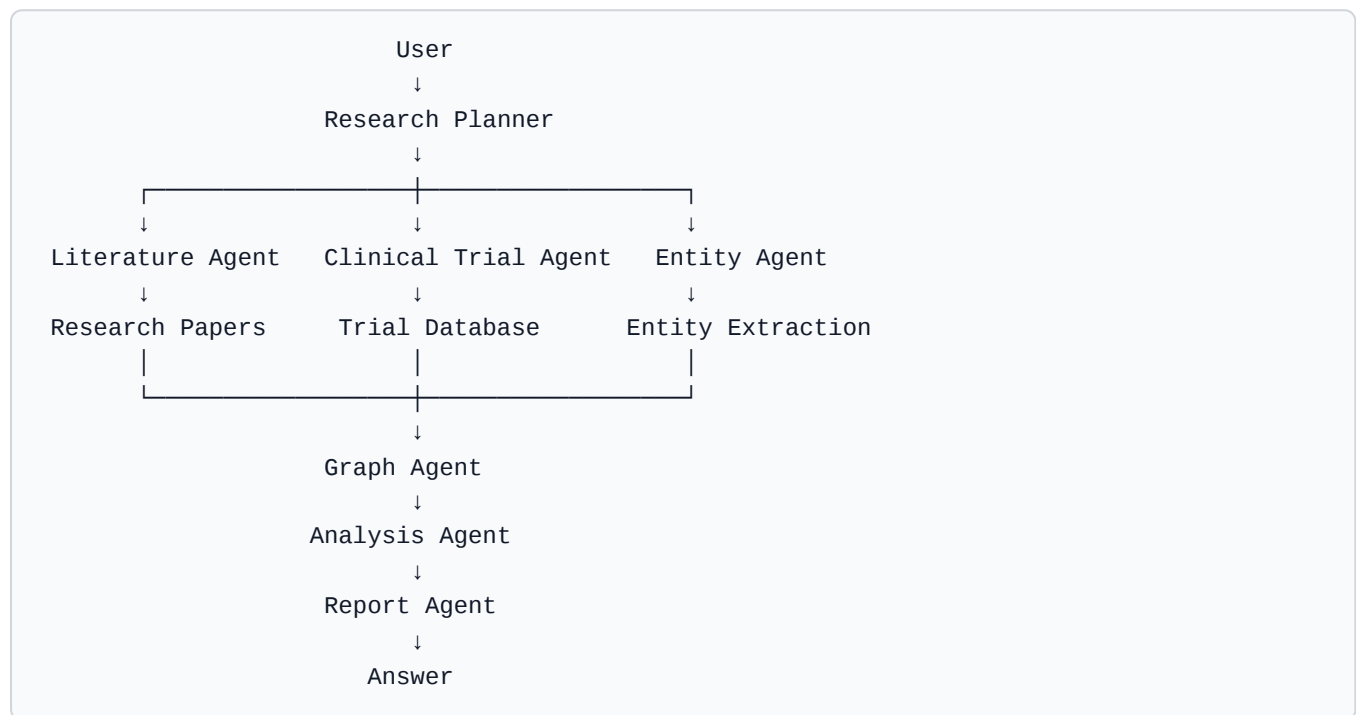
Biomedical Research Example

Consider a biomedical research system.

User request:

"Research the relationship between a drug and a particular disease."

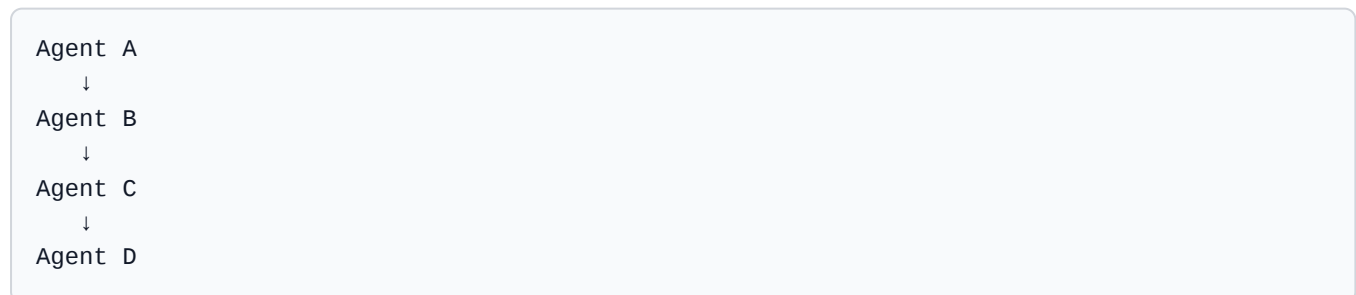
The workflow could be:



Each agent performs a different part of the research process.

Sequential Orchestration

In sequential orchestration, agents execute one after another.



The output of one agent becomes the input of another agent.

Sequential Research Example

Suppose we want to research a company.

```
graph TD;
  A[Search Agent] --> B[Extraction Agent];
  B --> C[Analysis Agent];
  C --> D[Summary Agent];
```

The workflow is:

```
graph TD;
  A[Search] --> B[Extract];
  B --> C[Analyze];
  C --> D[Summarize];
```

Each step depends on the previous step.

Software Development Example

A software development workflow can also be sequential.

```
graph TD;
  A[Requirement Agent] --> B[Design Agent];
  B --> C[Coding Agent];
  C --> D[Testing Agent];
  D --> E[Review Agent];
```

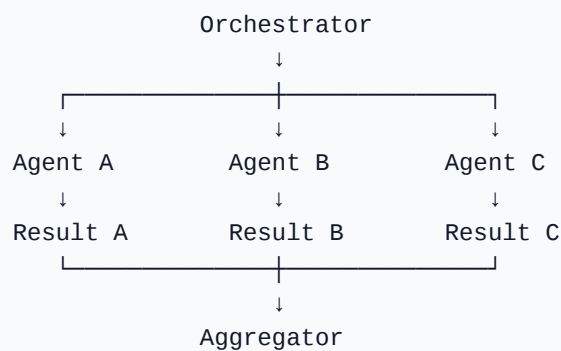
For example:

```
graph TD
    UR[User Requirement] --> B[Build a login API]
    B --> RA[Requirement Agent]
    RA --> ADA[API Design Agent]
    ADA --> CA[Code Agent]
    CA --> TA[Testing Agent]
    TA --> CRA[Code Review Agent]
```

This is useful when one stage needs the output of the previous stage.

Parallel Orchestration

Parallel orchestration means multiple agents execute at the same time.



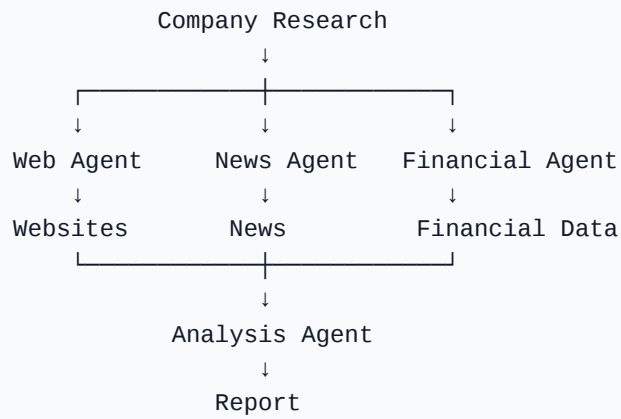
Parallel execution can reduce overall latency when tasks are independent.

Company Research Example

Suppose the user asks:

"Research this company."

The system can perform:



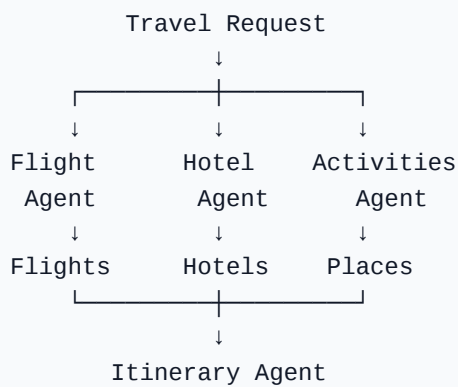
These tasks do not necessarily need to wait for each other.

Travel Research Example

For a travel request:

"Plan my trip to Delhi."

Independent tasks could execute in parallel:



Flights, hotels, and activities can be researched independently.

Sequential vs Parallel

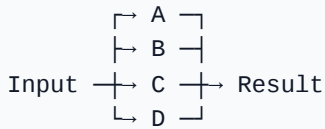
Sequential

A → B → C → D

Useful when:

- Each step depends on the previous step
- Output of A is required by B
- Output of B is required by C

Parallel

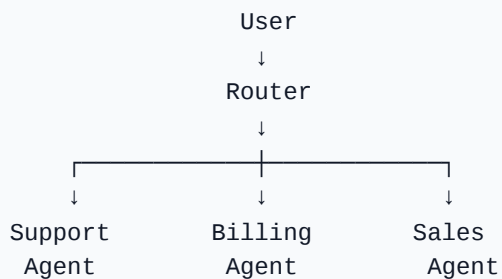


Useful when:

- Tasks are independent
- Multiple sources need to be queried
- Latency needs to be reduced

Router-Based Orchestration

A router decides which agent should handle the request.



The router does not necessarily perform the task itself.

It decides:

"Which agent is responsible for this request?"

IT Support Router Example

User:

"My VPN is not connecting."

Router:

```
graph TD
    Input --> Router
    Router --> IT_Support_Agent[IT Support Agent]
```

User:

"I need to reset my password."

Router:

```
graph TD
    Input --> Router
    Router --> Identity_Access_Agent[Identity / Access Agent]
```

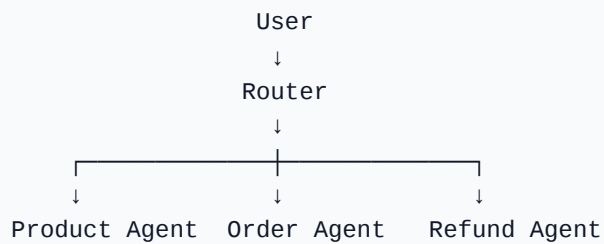
User:

"I need information about my laptop request."

Router:

```
graph TD
    Input --> Router
    Router --> Hardware_Asset_Agent[Hardware / Asset Agent]
```

E-Commerce Router Example



Examples:

"Find a laptop"



Product Agent

"Where is my order?"



Order Agent

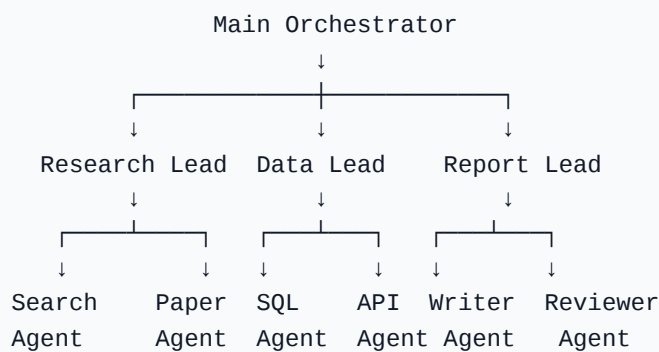
"I want a refund"



Refund Agent

Hierarchical Orchestration

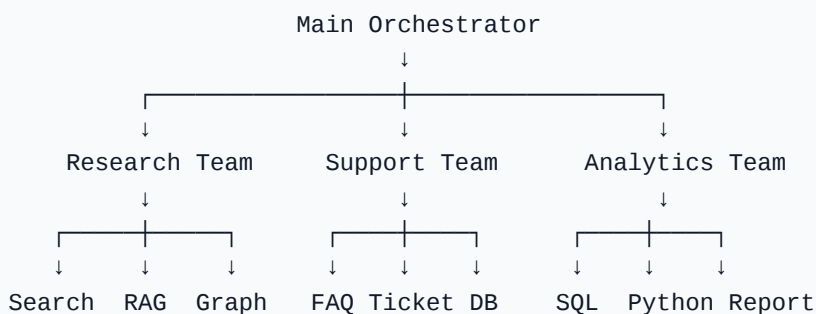
Hierarchical orchestration contains multiple levels of orchestration.



The top-level orchestrator coordinates specialized sub-orchestrators.

Enterprise AI Platform Example

An enterprise AI platform may have:



This architecture allows large systems to be decomposed into manageable groups.

Handoff

A handoff occurs when one agent transfers responsibility to another agent.

Example:

```
Customer Agent
  ↓
Detects technical issue
  ↓
Technical Support Agent
  ↓
Investigates issue
  ↓
Billing Agent
```

The first agent does not need to complete the entire workflow.

It can transfer control to the agent that is better suited for the next task.

Structured Handoff

A good handoff should contain structured information.

Instead of:

```
"Please handle this."
```

Use:

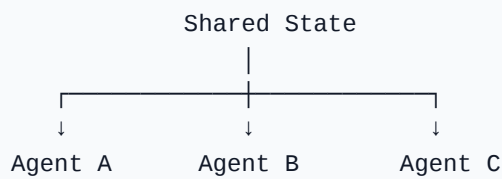
```
{
  "task": "Investigate VPN connection failure",
  "user_id": "12345",
  "issue": "VPN authentication fails",
  "previous_steps": [
    "Password reset completed",
    "VPN client restarted"
  ],
  "expected_action": "Diagnose VPN authentication issue"
}
```

This gives the next agent the context it needs.

Shared State

Multiple agents may need access to the same state.

For example:



Shared state can contain:

- User information
- Conversation history
- Task status
- Previous agent outputs
- Retrieved documents
- Tool results
- Errors
- Intermediate results

Why Shared State Matters

Without shared state:

```
Agent A
"I found this information."

Agent B
"What information?"
```

With shared state:

```
Agent A
  ↓
Stores result
  ↓
Shared State
  ↓
Agent B
  ↓
Uses result
```

Shared state reduces unnecessary repeated work.

Agent-to-Agent Communication

Agents need a reliable communication mechanism.

Communication can happen through:

```
graph TD; Messages --> Structured_JSON[Structured JSON]; Structured_JSON --> Shared_State[Shared State]; Shared_State --> Events; Events --> Database;
```

Messages
↓
Structured JSON
↓
Shared State
↓
Events
↓
Database

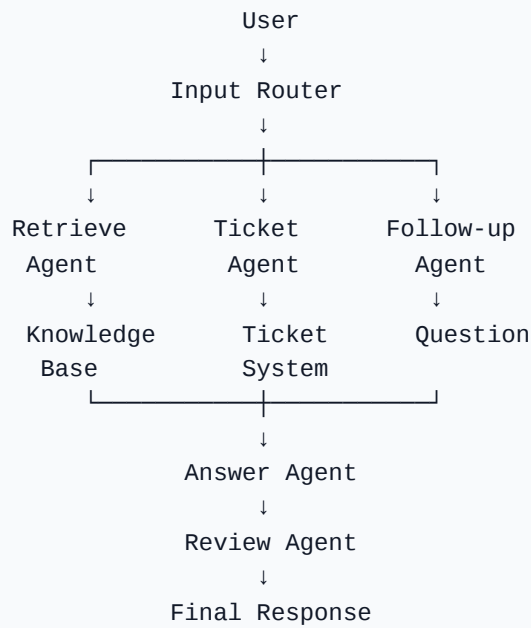
Example:

```
{
  "agent": "research_agent",
  "status": "completed",
  "result": {
    "papers_found": 12,
    "relevant_papers": 5
  }
}
```

Another agent can consume this structured result.

Complete IT Support Multi-Agent System

A production IT support system could look like:



The architecture separates responsibilities.

IT Support Workflow

Example request:

"My laptop cannot connect to the company VPN."

Possible workflow:

```
User
↓
Input Router
↓
Retrieve Agent
↓
Search Knowledge Base
↓
Relevant troubleshooting information
↓
Answer Agent
↓
Review Agent
↓
Final Response
```

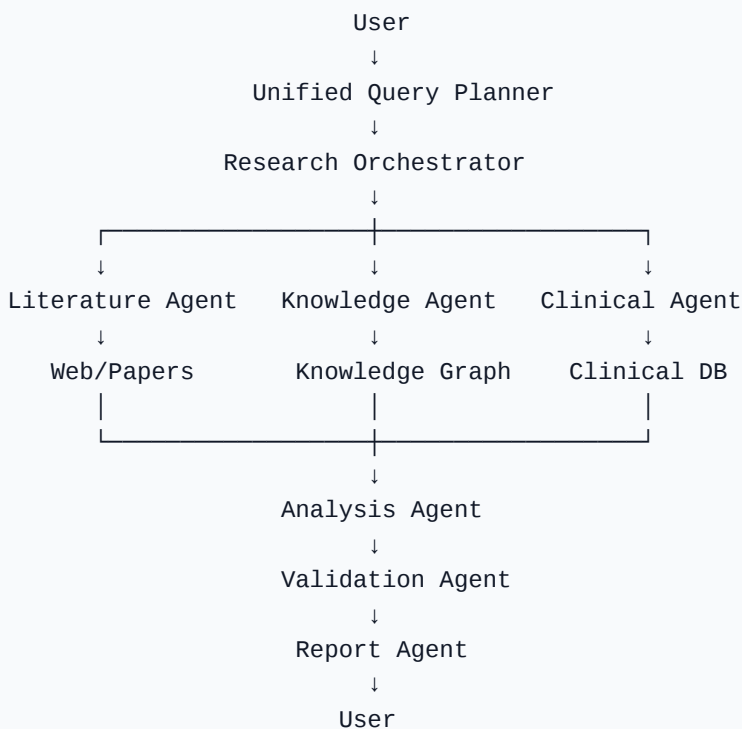
If the issue cannot be solved:

```
graph TD
    User --> Router
    Router --> RetrieveAgent[Retrieve Agent]
    RetrieveAgent --> NoSolution[No useful solution]
    NoSolution --> TicketAgent[Ticket Agent]
    TicketAgent --> CreateTicket[Create Support Ticket]
```

The workflow can therefore branch depending on the situation.

Research Agent Architecture

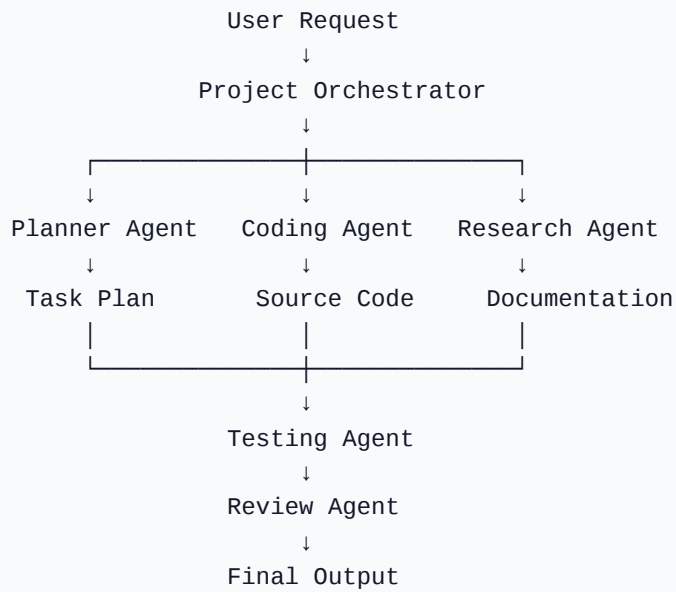
A complex research platform may use:



This architecture is useful when the research problem contains multiple independent and dependent stages.

Software Engineering Multi-Agent System

Another example is an AI software development platform.

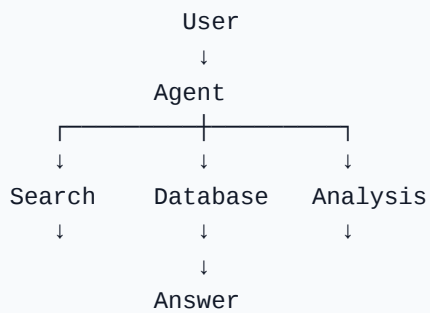


The Testing Agent can validate the work produced by the Coding Agent.

The Review Agent can inspect the final result.

One Agent vs Multi-Agent

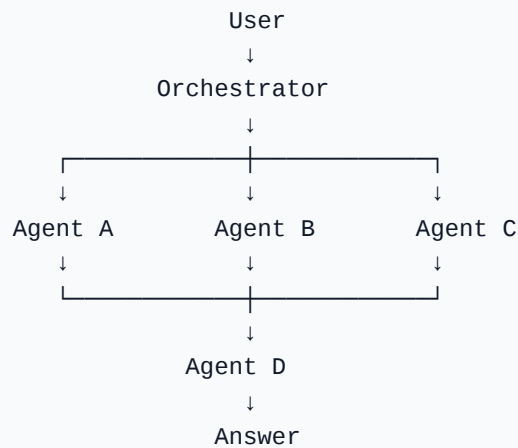
Single Agent



Advantages:

- Simpler
 - Easier to deploy
 - Easier to debug
 - Lower orchestration overhead
-

Multi-Agent



Advantages:

- Specialization
- Better separation of responsibilities
- Different tools per agent
- Different models per agent
- Complex workflows can be decomposed

But multi-agent systems also introduce additional complexity.

When NOT to Use Multi-Agent

Multi-agent does not automatically mean better.

If the task is:

"What is the capital of France?"

A multi-agent system would be unnecessary.

```
graph TD; User --> LLM; LLM --> Answer;
```

The diagram shows a simple single-agent workflow. A 'User' sends a request to a Large Language Model (LLM), which then provides the 'Answer'.

There is no reason to build:

```
Router
↓
Research Agent
↓
Validation Agent
↓
Summary Agent
↓
Review Agent
```

for a simple question.

The architecture should match the problem.

Multi-Agent Failure Modes

Wrong Routing

The router sends the request to the wrong agent.

```
User
↓
Router
↓
Wrong Agent
↓
Incorrect Workflow
```

Bad Handoff

Agent A provides incomplete information to Agent B.

```
Agent A
↓
Incomplete Context
↓
Agent B
↓
Incorrect Result
```

Structured handoffs reduce this problem.

Conflicting Agents

Two agents may produce different answers.

Agent A

↓

Result A

Agent B

↓

Result B

Result A $\neq$ Result B

A validation or reconciliation mechanism may be required.

Infinite Loops

Agents may repeatedly transfer tasks between each other.

Agent A

↓

Agent B

↓

Agent A

↓

Agent B

↓

Agent A

↓

...

Production systems should have:

- Maximum iterations
- Maximum handoffs
- Timeout limits
- Termination conditions

Duplicate Work

Multiple agents may perform the same task.

Agent A → Search database

Agent B → Search database

Agent C → Search database

This increases:

- Cost

- Latency
- Complexity

Shared state and clear responsibilities can reduce duplicate work.

State Conflicts

Multiple agents may modify shared state incorrectly.

Example:

```
Agent A:  
status = "completed"
```

```
Agent B:  
status = "failed"
```

The system needs clear ownership and state transition rules.

Cost Explosion

More agents mean more LLM calls.

Example:

```
1 Agent  
↓  
5 LLM calls
```

versus:

```
5 Agents  
↓  
Each makes 5 calls  
↓  
25 LLM calls
```

Without proper control, multi-agent systems can become expensive.

Useful techniques include:

- Smaller models for simple tasks
- Caching
- Context optimization
- Limiting agent iterations

- Tool result reuse
- Clear termination conditions

Latency

Sequential workflows can become slow.

Example:

```
Agent A → 2 seconds
Agent B → 3 seconds
Agent C → 4 seconds
Agent D → 2 seconds
```

Total:

```
2 + 3 + 4 + 2 = 11 seconds
```

If some tasks are independent, parallel execution can reduce total latency.

```
Agent A ┌
Agent B ┌
Agent C ┌ → Aggregator
Agent D ┌
```

Designing a Good Multi-Agent System

Start with the problem.

Do not start by asking:

"How many agents should I create?"

Instead ask:

"What responsibilities exist in this workflow?"

Then identify:

Responsibilities
↓
Specialized Agents
↓
Tools
↓
Communication
↓
Orchestration
↓
State Management
↓
Validation

Clear Agent Responsibilities

Each agent should have a clear purpose.

Good:

Research Agent
→ Research information

Good:

Ticket Agent
→ Create and manage tickets

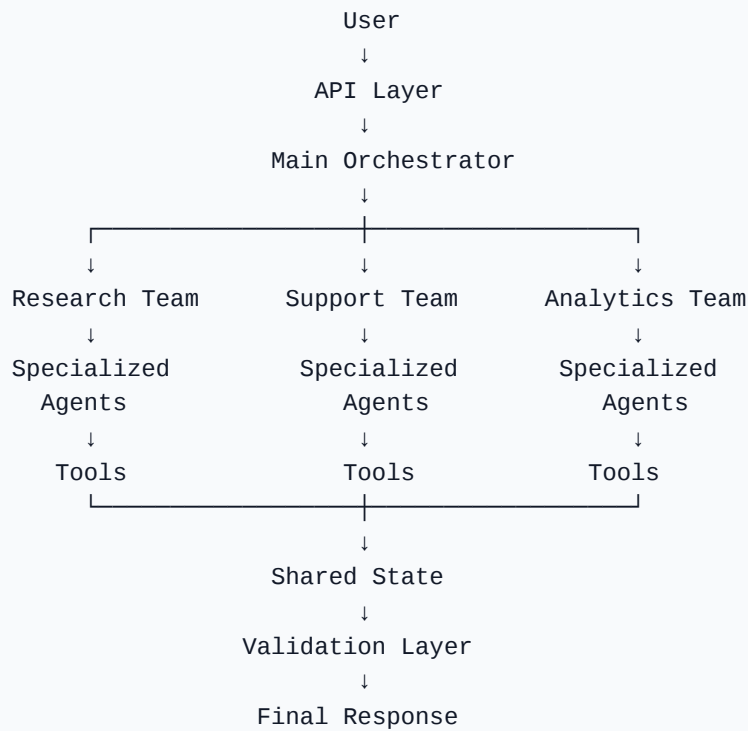
Less useful:

General Agent
→ Do everything

Clear boundaries make systems easier to maintain.

Real Production Architecture

A production-grade multi-agent platform may look like:



Around this architecture, production systems also need:

- Observability
- Logging
- Tracing
- Authentication
- Authorization
- Error Handling
- Retries
- Timeouts
- Evaluation
- Monitoring
- Cost Tracking

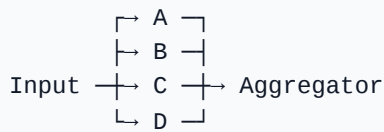
Four Major Orchestration Patterns

Sequential

A → B → C → D

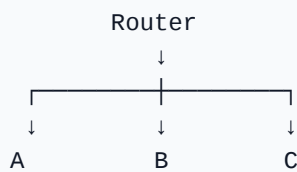
Use when each stage depends on the previous stage.

Parallel



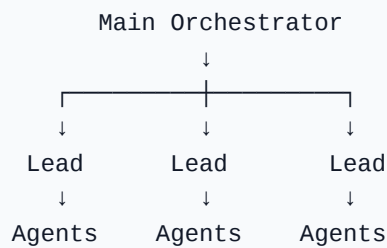
Use when tasks can execute independently.

Router



Use when different requests require different specialists.

Hierarchical

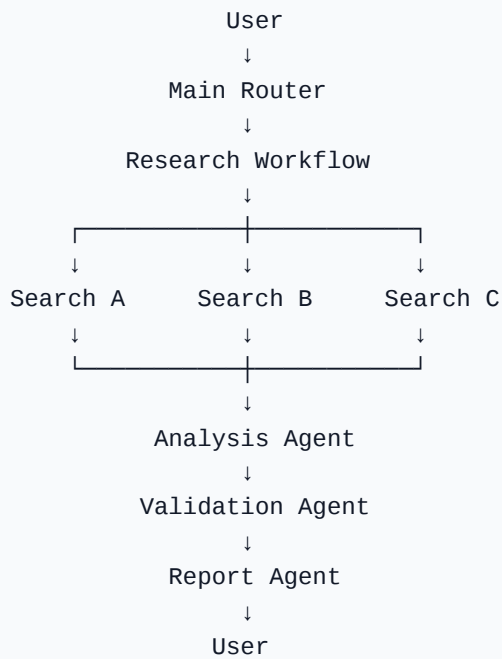


Use for large systems with multiple groups of responsibilities.

Putting Everything Together

A real system can combine multiple orchestration patterns.

For example:



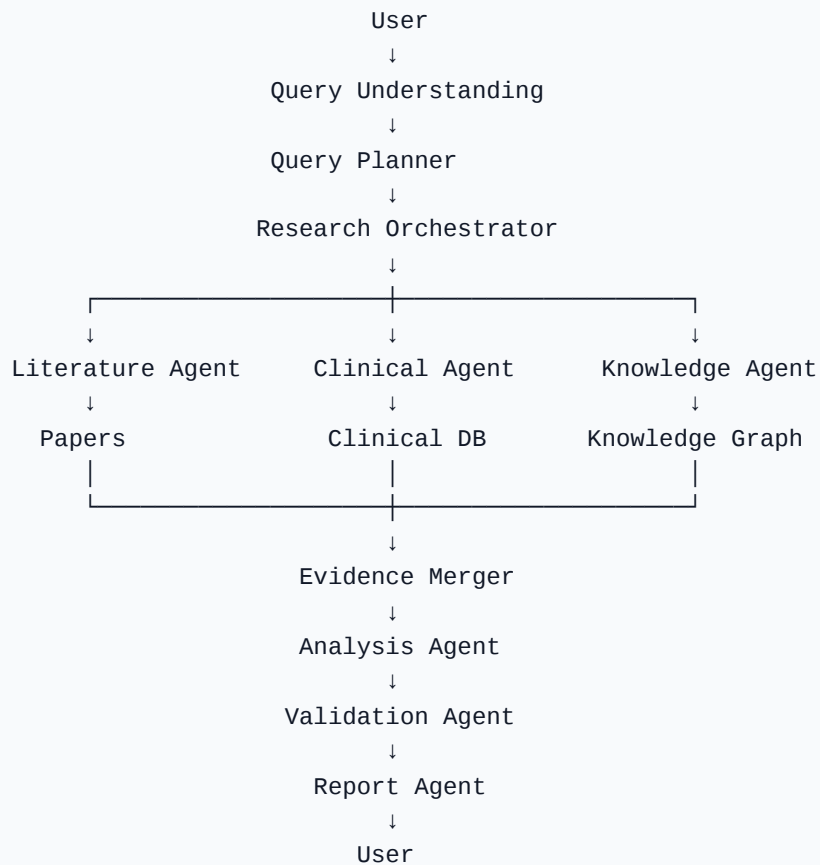
Here we have:

Router
+
Parallel
+
Sequential

Large agentic systems usually combine multiple patterns rather than using only one.

AI Research Platform Example

A complete research system might look like:



This architecture combines:

- Planning
- Routing
- Parallel execution
- Specialized agents
- Shared state
- Evidence aggregation
- Validation
- Final generation

Important Design Principle

The goal of multi-agent architecture is **not to create as many agents as possible**.

The goal is:

Create the right number of specialized components for the complexity of the problem.

Too few agents:

One huge agent
↓
Too many responsibilities
↓
Hard to maintain

Too many agents:

Many tiny agents
↓
Too much communication
↓
More latency
More cost
More complexity

The architecture should find the right balance.

Final Mental Model

Think about a company.

A company does not have one person doing everything.

Instead:

Manager
↓
Different Teams
↓
Different Specialists
↓
Different Responsibilities
↓
Collaboration
↓
Final Business Result

Multi-agent systems follow a similar idea.


```
Orchestrator
  ↓
Specialized Agents
  ↓
Tools
  ↓
Shared Information
  ↓
Collaboration
  ↓
Final Result
```

Final Takeaway

A single agent is useful for simple and moderately complex tasks.

Multi-agent systems become useful when a problem contains multiple clearly separable responsibilities.

The core concepts are:

```
Specialization
  ↓
Orchestration
  ↓
Communication
  ↓
Shared State
  ↓
Coordination
  ↓
Validation
  ↓
Final Result
```

The four major orchestration patterns are:

```
Sequential
Parallel
Router
Hierarchical
```

And production systems may combine them.

The most important principle is:

Use multi-agent architecture when decomposition provides a real benefit, not simply because multiple agents are possible.

Core Concepts

Concept	Meaning
Agent	Performs a specific task
Specialized Agent	Agent focused on a particular responsibility
Orchestrator	Coordinates agents and workflow
Router	Selects the appropriate agent
Sequential	Agents execute one after another
Parallel	Independent agents execute simultaneously
Hierarchical	Multiple levels of orchestration
Handoff	Transfers responsibility between agents
Shared State	Common information available to agents
Agent-to-Agent Communication	Agents exchange information
Aggregator	Combines outputs from multiple agents
Validation Agent	Checks results
Termination Condition	Determines when the workflow should stop
Multi-Agent System	Multiple agents collaborating on a larger task
...	